

Análisis de Optimalidad:

1. Máquina BFS/DFS:

- ¿Por qué este algoritmo casi siempre genera la peor ganancia?

BFS y DFS ignoran por completo el peso real de cada tramo. Para ellos todas las aristas cuestan lo mismo, por lo que buscan rutas con menos pasos, no con menor costo. En un grafo donde los pesos varían mucho, esto puede hacer que el algoritmo prefiera tramos extremadamente caros solo porque reducen el número de pasos, generando así las peores ganancias.

- ¿Garantiza BFS o DFS encontrar algún camino? Sí/No y por qué

Sí, siempre que el camino exista. BFS y DFS exploran todos los nodos alcanzables desde el inicio. Si la meta está en esa región alcanzable, ambos algoritmos garantizan encontrar un camino. Si no existe, terminarán la exploración sin encontrarlo.

- Explique cómo la ignorancia de los pesos lleva a rutas costosas.

Como BFS y DFS ignoran los pesos, solo consideran la cantidad de pasos para llegar a la meta. Por ejemplo, si existe un camino de 4 nodos con costo total 40 y otro de 3 nodos con costo 70, BFS preferirá el segundo por requerir menos pasos, aunque sea más caro. Al no evaluar costos, pueden terminar construyendo rutas muy costosas.

- Muestre una captura de pantalla de una tubería nivel 1, que tome un camino visiblemente absurdo y costoso en su mapa.

Prefiere el camino más corto en cantidad de nodos en vez del más liviano.



2. Máquina Greedy (Prim):

- **¿Por qué este algoritmo es mejor que el BFS/DFS, pero aun así no es perfecto?**

A diferencia de BFS/DFS, Prim sí toma decisiones basadas en los pesos, por lo que evita tramos muy costosos y genera rutas más razonables. Sin embargo, Prim intenta construir un árbol generador mínimo, no el camino mínimo entre dos nodos. Por eso puede tomar decisiones locales baratas que no garantizan que el camino entre inicio y meta sea el más barato posible.

- **Describa y compare los conceptos de óptimo local y óptimo global.**

Óptimo local: se refiere a la mejor opción inmediata, es decir entre todas las alternativas posibles alrededor mío cuál es la mejor, no analiza lo que viene después de esa opción.

Óptimo global: se refiere a la mejor opción entre todas las posibles, es decir considerando todo el camino, no sólo las inmediatas.

Prim busca un óptimo local porque elige la arista más barata disponible en cada paso sin considerar cómo afectará el resto del camino. Esto puede llevar a un camino globalmente caro, ya que un tramo barato ahora podría obligar a usar tramos carísimos más adelante. El óptimo global sería el camino de menor costo total entre inicio y meta, que Prim no garantiza.

- **Diseñe o encuentre un escenario en su mapa donde el algoritmo greedy tome una mala decisión.**

Se tiene el grafo:

A C 3
C D 3
A B 10
B E 20
D E 80

Greedy falla cuando una arista localmente barata fuerza al algoritmo a atravesar tramos extremadamente costosos más adelante. En el ejemplo, Greedy elige A–C (3) y luego C–D (3), pero esa ruta obliga a tomar más adelante el tramo D–E (80). El camino global óptimo era A–B–E (10 + 20), pero Greedy nunca lo ve porque solo optimiza localmente.

3.Máquina Dijkstra/A*:

- **¿Por qué este algoritmo siempre encuentra el camino más rentable?**

Dijkstra y A* siempre encuentran el camino más rentable porque expanden primero los nodos cuyo costo acumulado desde el inicio es mínimo. Esto garantiza que nunca se avance por una ruta más costosa si existe otra más barata disponible. Así, la primera vez que se alcanza la meta, el camino encontrado es el de menor costo total.

- **Explique cómo explora el grafo en comparación con los otros dos.**

Dijkstra/A* avanzan siempre por el camino que va acumulando el menor costo total. BFS avanza por cantidad de pasos ignorando precios, DFS se mete profundo sin analizar costos, y Prim elige solo la arista más barata del momento sin considerar el camino completo. Dijkstra/A* exploran de forma más inteligente, expandiendo primero las rutas más económicas hasta encontrar el costo mínimo total.

Análisis de Complejidad Temporal

- **Declare la complejidad temporal (Big O) de BFS, DFS y Dijkstra en función de los vértices y los aristas.**

Para BFS y el DFS la complejidad temporal es de $O(V+E)$ ya que cada vértice y cada arista se procesan una sola vez. Dijkstra depende de la estructura usada en para elegir el siguiente nodo con menor costo.

- **Si usó una cola de prioridad (Dijkstra), ¿Cómo afecta esto la complejidad en comparación con una búsqueda simple en un arreglo?**

Si se usa una cola de prioridad la complejidad va a ser de $O((V+E)\log V)$ ya que cada extracción del vértice con menor distancia y cada actualización de distancia cuestan $\log V$. Mientras que si se usa un arreglo simple, la complejidad es de $O(V^2+E)$, ya que seleccionar el vértice de distancia mínima cuesta $O(V)$ cada vez.

- **Si Dijkstra es más lento de calcular, ¿por qué se utiliza?**

Aunque Dijkstra es más lento, es el único algoritmo de los implementados que garantiza el camino más barato posible. BFS y DFS ignoran pesos, y Prim puede tomar decisiones locales que no producen el mejor camino. La precisión de Dijkstra en minimizar el costo total justifica su mayor tiempo de cómputo.

Análisis de Estructuras de Datos

- **¿Cómo influye la densidad del mapa (cantidad de aristas) en la elección entre una matriz y una lista de adyacencia?**

La densidad determina qué estructura usa la memoria y el tiempo de manera más eficiente.

En un grafo poco denso, una lista de adyacencia es mejor porque solo almacena las conexiones reales y evita guardar miles de enlaces inexistentes.

En un grafo muy denso, una matriz de adyacencia sería preferible porque permite acceso inmediato a cualquier conexión, aunque consuma más memoria.

- **¿Cómo habría cambiado el rendimiento (memoria/velocidad) de los algoritmos si hubieran elegido la otra estructura?**

Si se hubiera usado una matriz de adyacencia, el consumo de memoria habría sido muchísimo mayor, ya que habría que almacenar V^2 posiciones aunque la mayoría de conexiones no existen. Además, algoritmos como Dijkstra y BFS habrían sido más lentos porque tendrían que revisar todos los posibles vecinos de cada nodo, no solo los que realmente existen.